PIM developer manual

Environment and launch

This command-line personal information manager was developed and tested on macOS using Python 3.13.0. It requires Python 3.10 or later. The application, tests and coverage tool use only the Python standard library. No installation, package download, network access or compilation step is required.

Open the pim folder in Visual Studio Code or any Python-capable IDE. No IDE-specific configuration is required. Set the working directory to the pim folder and the entry point to main.py. The commands below run in Terminal from that folder.

```
python3 main.py
python3 main.py --file "my-records.pim"
python3 main.py --script examples/demo.txt
```

The first command starts an empty interactive session. The second loads an existing file before starting. The third runs the supplied demonstration and writes/replaces demo.pim in the current working directory. Relative data paths resolve against the current working directory, not the script's folder.

Build and debugging

Python runs the source directly. For a syntax compilation check and debugging with the standard-library debugger:

```
python3 -m compileall -q main.py model controller view tests tools
python3 -m pdb main.py
```

In pdb, enter a breakpoint such as b controller/commands.py:50 and then c to continue. Alternatively, set breakpoints in an IDE and launch main.py with no arguments. Use --script examples/demo.txt as program arguments to debug a reproducible session. No environment variables or credentials are needed.

Code structure

The program follows Model-View-Controller separation. main.py handles input/output and application startup. Controller.execute translates command strings into model calls and produces a CommandResult. The view formats immutable records and provides help text. Model code imports neither controller nor view and performs no terminal interaction.

Component	Responsibility
model.records	Record, Note, Task, Event, Contact; construction and update validation; time normalization
model.criteria	Predicate, Negation, Junction and Parser; validated Boolean expression trees
model.manager	PIM collection; creation, lookup, update, deletion, search and snapshots
model.storage	Versioned UTF-8 JSON persistence; atomic file replacement; complete validation on load
model.errors	PIMError and expected validation, lookup, criterion and storage exceptions
controller.commands	CommandResult and Controller; tokenization, dispatch and dirty state

Component	Responsibility
view.terminal	Help text and detailed record formatting

Record is a frozen dataclass. Its subclasses add required fields. Updating builds and validates a replacement record before changing the collection. PIM owns the ID allocator and returns a tuple of records sorted by ID. Callers cannot mutate a returned record to bypass validation.

The parser tokenizes the criterion and builds immutable predicates and logical nodes. It never uses eval, exec, shell commands or pickle. Precedence is parentheses, negation, conjunction, then disjunction. Junctions short-circuit. Expression length and nesting limits prevent excessive recursive parsing.

Saving serializes a snapshot to a temporary file in the destination directory, flushes and fsyncs it, then uses os.replace to replace the destination. Loading creates a separate PIM instance and validates the complete document before the controller replaces its active instance. An invalid file therefore cannot partially replace current records. The format preserves next_id so deleted IDs are not reused after reload.

Principal model API

Public operation	Return and expected failures
PIM.create(kind, **data)	Record; ValidationError for unsupported types or invalid fields
PIM.get(record_id)	Record; ValidationError for invalid ID, RecordNotFound for missing ID
PIM.all()	tuple of Record sorted by ID
PIM.update(record_id, **changes)	Replacement Record; lookup or validation errors; ID/type changes rejected
PIM.delete(record_id)	Removed Record; lookup errors
PIM.search(expression)	tuple of matching Record; CriteriaError for invalid expressions
PIM.snapshot()	Detached dict containing format, version, next_id and records
PIM.from_snapshot(snapshot)	New PIM; ValidationError for invalid structure or records
model.save(pim, path)	None; StorageError if validation of extension or writing fails
model.load(path)	New PIM; StorageError if reading, decoding or validation fails
model.parse_time(value)	datetime normalized to +08:00; ValidationError for invalid time

The record creation/update APIs use keyword fields: text; description/deadline; description/start/alarm; or name/address/mobile. The method's record type and target record ID are positional-only, so fields with conflicting names are rejected as invalid data rather than causing an argument-binding crash.

Verification and coverage

```
python3 -m unittest discover -s tests -v
python3 tools/measure_coverage.py
```

Tests use unittest, temporary directories and unittest.mock. They exercise each user story; every searchable text and time field; all comparison/logical operators; malformed expressions; invalid updates; persistence round trips; corrupt files; duplicate keys/IDs; non-reused IDs; failed writes; unsaved changes; and a real-process

demonstration run. Files created by tests are isolated in temporary directories.

The coverage script runs the tests in a fresh process with trace, includes every model source module, and writes TEST_COVERAGE.md plus annotated .coverage files. A line prefixed with >>>>>> in an annotated file was not executed. Reported line coverage is not branch coverage or proof that every input is correct. TEST_COVERAGE.pdf records the supplied build's measurement and must be refreshed after changes.

Scope and maintenance

The application is single-user and single-process. Do not concurrently save from multiple instances to the same destination. It has no database, encryption, network synchronization, GUI or background alarm service. File saving is explicit; quitting and loading guard against accidentally discarding unsaved changes. The format is version 1 and rejects unknown versions and fields instead of guessing.

OpenAI Codex assisted with source code, tests and documentation. No existing group SRS was supplied; the implementation rules in the user manual and requirements coverage report need to match the group's final SRS. The group must complete its own Honour Declaration and remaining non-coding deliverables.